RISC V Registers

- There are 32 registers each of 32 bits.
- Registers are numbered 0 to 31
- They are given a special name as well to indicate a register's conventional purpose.

Name	Register Number	Use
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0–2	x5–7	Temporary registers
s0/fp	x8	Saved register / Frame pointer
s1	x9	Saved register
a0–1	x10–11	Function arguments / Return values
a2–7	x12–17	Function arguments
s2–11	x18–27	Saved registers
t3-6	x28–31	Temporary registers

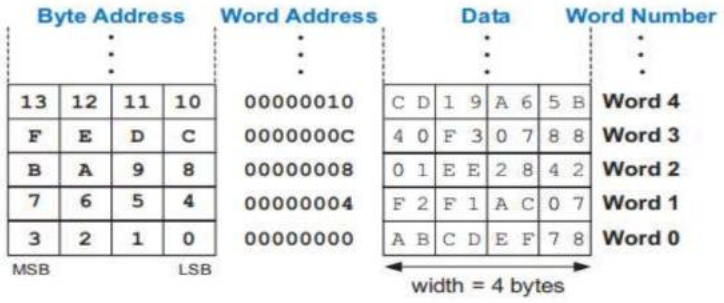
Registers

- **a0- a7** → (8 argument registers)
- Used to hold the parameters passed from the ***caller*** procedure to the ***callee*** procedure
- **a0, a1** → argument + return value register
- **s0- s11→ (callee saved registers)** If a procedure(callee) wants to make use of these registers, then it must preserve* their contents, before using it.
- **t0- t6→ (caller saved registers)** Caller function needs to preserve* their contents, before calling some other procedure. Caller should retrieve their values after returning from the procedure.
- ***Hint:** Push over stack to preserve contents.

Memory Operands

- To access a word in memory, the instruction must supply the memory address.
- The RV32I RISC-V architecture uses 32-bit memory addresses and 32-bit data words.
- Data is stored in little endian way. Little part of data is stored first.
- Example: store 0xA1B2C3D4 at memory address 0x1000

Note that the most significant byte (MSB) is on the left and the least significant byte (LSB) is on the right.



Standard Convention Used in This Course

Decimal term	Abbreviation	Value	Binary term	Abbreviation	Value	% Larger
kilobyte	KB	10^3	kibibyte	KiB	2^{10}	2%
megabyte	MB	10^6	mebibyte	MiB	2^{20}	5%
gigabyte	GB	10^9	gibibyte	GiB	2^{30}	7%
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}	10%
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}	13%
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}	15%
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}	18%
yottabyte	YB	10^{24}	yobibyte	YiB	2^{80}	21%
ronnabyte	RB	10^{27}	robibyte	RiB	2^{90}	24%
queccabyte	QB	10^{30}	quebibyte	QiB	2^{100}	27%

Note: The 2^X vs. 10^Y bytes ambiguity was resolved by adding a binary notation for all the common size terms. In the last column we note how much larger the binary term is than its corresponding decimal term, which is compounded as we head down the chart. These prefixes work for bits as well as bytes, so gigabit (Gb) is 10^9 bits while gibibits (Gib) is 2^{30} bits.

RV32I Instruction Formats

- 40 core instructions

Name (Field Size)	Field 7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	Comments
R-type	funct7	rs2	rs1	funct3	rd	opcode	Arithmetic instruction format
I-type	immediate[11:0]		rs1	funct3	rd	opcode	Loads & immediate arithmetic
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode	Stores
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode	Conditional branch format
UJ-type	immediate[20,10:1,11,19:12]				rd	opcode	Unconditional jump format
U-type	immediate[31:12]				rd	opcode	Upper immediate format

R (Register) Format

7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
funct7	**rs2**	**rs1**	**funct3**	**rd**	**opcode**

- **opcode:** denotes the operation and format of the instruction. All R-type instructions have op = 011 0011 = 0x33 = $51)_{10}$
- **rs1** : identifier of first source register
- **rs2** : identifier of second source register
- **rd :** identifier of destination register
- **funct7 :** additional opcode field, distinguishes among R-type instructions
- **funct3 :** additional opcode field, distinguishes among R-type instructions

RV32I: R-Type Instructions

op	funct3	funct7	Type	Instruction	Description	Operation
0110011 (51)	000	0000000	R	add rd, rs1, rs2	add	rd = rs1 + rs2
0110011 (51)	000	0100000	R	sub rd, rs1, rs2	sub	rd = rs1 − rs2
0110011 (51)	001	0000000	R	sll rd, rs1, rs2	shift left logical	rd = rs1 << $rs2_{4:0}$
0110011 (51)	010	0000000	R	slt rd, rs1, rs2	set less than	rd = (rs1 < rs2)
0110011 (51)	011	0000000	R	sltu rd, rs1, rs2	set less than unsigned	rd = (rs1 < rs2)
0110011 (51)	100	0000000	R	xor rd, rs1, rs2	xor	rd = rs1 ^ rs2
0110011 (51)	101	0000000	R	srl rd, rs1, rs2	shift right logical	rd = rs1 >> $rs2_{4:0}$
0110011 (51)	101	0100000	R	sra rd, rs1, rs2	shift right arithmetic	rd = rs1 >>> $rs2_{4:0}$
0110011 (51)	110	0000000	R	or rd, rs1, rs2	or	rd = rs1 | rs2
0110011 (51)	111	0000000	R	and rd, rs1, rs2	and	rd = rs1 & rs2

I (Immediate) Format

12 bits	5 bits	3 bits	5 bits	7 bits
Immediate [11:0]	**rs1**	**funct3**	**rd**	**opcode**

- Both lw & addi use **I-Format**
- **opcode:** denotes the operation
- **rs1** : **base register** identifier in case of **load** instructions and first source register identifier in all other instructions using I format (e.g. addi, andi, ori etc.)
- **rd : destination** register identifier
- **immediate:** 12-bit signed offset expressed in bytes (*expressed in decimal here*)

RV32I: I-type Instructions

op	funct3	funct7	Type	Instruction	Description	Operation
0000011 (3)	000	–	I	`lb rd, imm(rs1)`	load byte	`rd = SignExt([Address]7:0)`
0000011 (3)	001	–	I	`lh rd, imm(rs1)`	load half	`rd = SignExt([Address]15:0)`
0000011 (3)	010	–	I	`lw rd, imm(rs1)`	load word	`rd = [Address]31:0`
0000011 (3)	100	–	I	`lbu rd, imm(rs1)`	load byte unsigned	`rd = ZeroExt([Address]7:0)`
0000011 (3)	101	–	I	`lhu rd, imm(rs1)`	load half unsigned	`rd = ZeroExt([Address]15:0)`
0010011 (19)	000	–	I	`addi rd, rs1, imm`	add immediate	`rd = rs1 + SignExt(imm)`
0010011 (19)	001	0000000*	I	`slli rd, rs1, uimm`	shift left logical immediate	`rd = rs1 << uimm`
0010011 (19)	010	–	I	`slti rd, rs1, imm`	set less than immediate	`rd = (rs1 < SignExt(imm))`
0010011 (19)	011	–	I	`sltiu rd, rs1, imm`	set less than imm. unsigned	`rd = (rs1 < SignExt(imm))`
0010011 (19)	100	–	I	`xori rd, rs1, imm`	xor immediate	`rd = rs1 ^ SignExt(imm)`
0010011 (19)	101	0000000*	I	`srli rd, rs1, uimm`	shift right logical immediate	`rd = rs1 >> uimm`
0010011 (19)	101	0100000*	I	`srai rd, rs1, uimm`	shift right arithmetic imm.	`rd = rs1 >>> uimm`
0010011 (19)	110	–	I	`ori rd, rs1, imm`	or immediate	`rd = rs1 | SignExt(imm)`
0010011 (19)	111	–	I	`andi rd, rs1, imm`	and immediate	`rd = rs1 & SignExt(imm)`
1100111 (103)	000	–	I	`jalr rd, rs1, imm`	jump and link register	`PC = rs1 + SignExt(imm), rd = PC + 4`

S (Store) Format

7	5	5	3	5	7
Immediate[11:5]	rs2	rs1	funct3	immediate [4:0]	opcode

- sw uses **S-Format**
- **opcode:** denotes the operation, opcode =35 for S–type instructions
- **rs1** : **base register** identifier for **store** instructions
- **rs2 : destination** register identifier
- **immediate:** 12-bit signed offset expressed in bytes (*expressed in decimal here*) – split into two fields.
- **funct3 :** distinguishes among various S-type instructions

RV32I: S-type Instructions

op	funct3	funct7	Type	Instruction	Description	Operation
0100011 (35)	000	–	S	sb rs2, imm(rs1)	store byte	$[Address]_{7:0} = rs2_{7:0}$
0100011 (35)	001	–	S	sh rs2, imm(rs1)	store half	$[Address]_{15:0} = rs2_{15:0}$
0100011 (35)	010	–	S	sw rs2, imm(rs1)	store word	$[Address]_{31:0} = rs2$

SB (Conditional Branch) Format

7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
Immediate [12, 10:5]	rs2	rs1	funct3	Immediate [4:1, 11]	opcode

- **opcode:** denotes the operation and format of the instruction. All SB-type instructions have op = 110 0011 = 0x63 = $99)_{10}$
- **rs1** : identifier of first source register
- **rs2** : identifier of second source register
- **funct3:** to distinguish among various SB type instructions
- **Immediate [12, 10:5] :** label/offset in bytes
- **Immediate [4:1, 11]:** label/offset in bytes
- **Note: bit 0 is not saved**

Conditional Branch Instructions

op	funct3	funct7	Type	Instruction	Description	Operation
1100011 (99)	000	–	B	`beq rs1, rs2, label`	branch if =	`if (rs1 == rs2) PC = BTA`
1100011 (99)	001	–	B	`bne rs1, rs2, label`	branch if ≠	`if (rs1 ≠ rs2) PC = BTA`
1100011 (99)	100	–	B	`blt rs1, rs2, label`	branch if <	`if (rs1 < rs2) PC = BTA`
1100011 (99)	101	–	B	`bge rs1, rs2, label`	branch if ≥	`if (rs1 ≥ rs2) PC = BTA`
1100011 (99)	110	–	B	`bltu rs1, rs2, label`	branch if < unsigned	`if (rs1 < rs2) PC = BTA`
1100011 (99)	111	–	B	`bgeu rs1, rs2, label`	branch if ≥ unsigned	`if (rs1 ≥ rs2) PC = BTA`

Satck Operation : 1. PUSH

- Saving an item on stack is regarded as ***push*** operation
- Stack pointer (sp) gets *decremented* with every push
- Pushing a register **s0** onto stack requires the following steps

```
addi    sp, sp, - 4          # decrement stack pointer by 4
sw      s0,  0(sp)           # copy the contents of s0 to new TOS
```


Satck Operation : **2. POP**

- Retrieving data from top of the stack is regarded as *pop* operation
- stack pointer (sp) gets in*cremented* with every pop
- popping a register **s0** off stack requires the following steps

```
lw    s0, 0(sp)        # copy the contents of TOS to s0
addi  sp, sp, 4        # increment stack pointer by 4
```

UJ (Unconditional Jump) Format

20 bits	5 bits	7 bits
Immediate [20, 10:1, 11, 19:12]	rd	Opcode

- **Opcode:** operation code 110 1111 $=111)_{10}$ for UJ format
- **rd:** register used to hold address of instruction following jal
- **Immediate:** most significant 20 bits of a 21-bit immediate jump offset. Bit 0 is not saved. This offset is used to construct jump target address (JTA)
- JTA = PC + (Immediate (20 bits) || 0)
- This JTA is loaded into PC . i.e. PC ← Jump Target Address

Function Call

jal rd, label

- **jal** (jump-and-link) performs two functions;

1. It stores return address (the address of instruction following the ***calling instruction)*** in register **rd** (i.e. rd = PC +4) &

2. It transfers control to the **label** (i.e. PC = Jump Target Address)

It follows UJ (Unconditional Jump) format.

Note: Alternate pseudo instruction is `jal label`

Returning from Procedure

- Use **Jump and Link Register – jalr** as last instruction in *each* procedure
- **Syntax:** `jalr rd, rs1, Imm` # PC = rs1 + Imm, rd = PC + 4
- It transfers control to the address contained in **rs1**
- Use as `jalr x0, ra, 0` # PC = ra + 0, rd is hardwired to 0
- It uses I – Format
- Alternate pseudo instruction is `ret` (return instruction)
- It performs an unconditional jump to the address contained in register **ra**.

1. The j (Jump) Pseudo-instruction

- **Instruction:** j label
- **Translation:** The assembler translates `j label` into `jal x0, label`
- **Function:** It performs an unconditional, PC-relative jump to the specified target address. Because it uses x0 as the destination register (which is hardwired to zero), it does not save the return address.
- Better than beq x0,x0, label
- With j label, label field is 20 bits wide. 21-bit label can be stored.
- **Note:** See other pseudo instructions at page 3 of Digital Design and Computer Architecture.

U (Upper Immediate) Format

20 bits	5 bits	7 bits
Immediate [31:12]	rd	Opcode

- **Opcode:** operation code
- **rd:** destination register identifier
- **Immediate:** most significant 20 bits [31:12] of a 32-bit immediate value is placed here. Least significant 21 bits [11:0] are always 0.

Opcodes:

- **LUI (Load Upper Immediate):** 0110111
- **AUIPC (Add Upper Immediate to PC):** 0010111

U – type instructions

1. LUI (Load Upper Immediate)

- Syntax: `lui rd, imm`
- It loads the upper 20 bits of the immediate value into the destination register rd, effectively creating a 32-bit constant with the lower 12 bits as zeros (rd = imm << 12).
- This instruction is used for creating large immediate value.

2. AUIPC (Add Upper Immediate to PC)

- Syntax: `auipc rd, imm`
- It adds the upper 20 bits of the immediate value (shifted left by 12) to the current Program Counter (PC) and stores the result in rd (rd = PC + (imm << 12)).
- This instruction is used for creating large PC- relative address.